TEMAS 3 Y 4: CIRCUITOS COMBINACIONALES I Y II.

SISTEMAS DIGITALES

Índice:

- Introducción.
- Síntesis de circuitos combinacionales sencillos utilizando circuitos integrados SSI.
- Análisis de circuitos combinacionales utilizando circuitos integrados SSI.
- Introducción a los bloques funcionales combinacionales (comerciales).
- Circuitos combinacionales MSI.
    - Decodificadores y demultiplexores.
    - Codificadores.
    - Multiplexores.
    - Comparadores.
    - Generadores / detectores de paridad.
    - Convertidores de código.
    - Circuitos aritméticos.
- Análisis y síntesis de circuitos combinacionales utilizando circuitos integrados SSI y MSI.

Introducción.

Hasta ahora hemos visto:

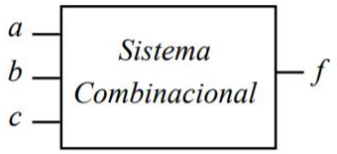
1. El sistema de numeración binario.
2. Algunos códigos binarios que se utilizan en el campo de la Electrónica Digital.
3. Un conjunto de axiomas que definen las Álgebras de Boole y algunos teoremas que nos permiten operar con variables y con funciones lógicas.
4. Los conceptos de constante, variable y función lógica; así como diversas formas de representar/definir una función lógica.
5. Las puertas lógicas (circuitos electrónicos).
6. Método de Karnaugh-Veitch para simplificar funciones dependientes de 2, 3 y 4 variables.

Todos estos conceptos y herramientas nos van a permitir analizar y diseñar circuitos digitales en lo que resta de curso.

El primer paso que es necesario dar en este sentido es clasificar los Sistemas Digitales de acuerdo con su comportamiento:

$$\textit{Sistemas Digitales}\begin{cases}\cdot\ \textit{Sistemas combinacionales.}\\ \cdot\ \textit{Sistemas secuenciales.}\end{cases}$$

DEF.: Se dice que un circuito dado es un sistema combinacional si el valor lógico de sus salidas en cualquier instante de tiempo t_1 que se considere, sólo depende del valor lógico de sus entradas en dicho instante de tiempo t_1. Ejemplo:

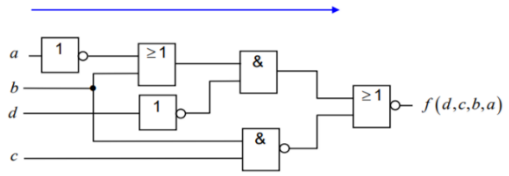


c	b	a	f
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

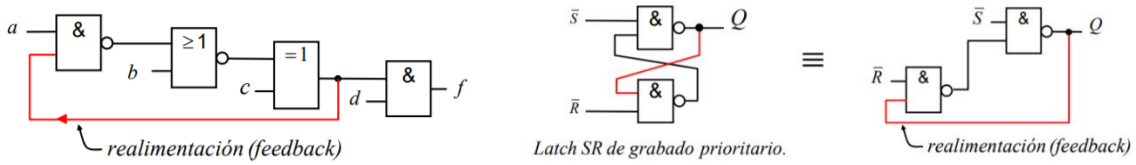
Nota: siempre que (c,b,a) = 1 1 0, la salida (f) será igual a 1.

Las puertas lógicas son circuitos combinacionales.

En general, se puede afirmar que cualquier circuito realizado con puertas lógicas que no tenga ninguna realimentación es un sistema combinacional.



Ejemplos de circuitos que no son combinacionales (son secuenciales):



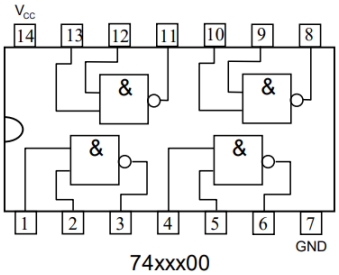
Teniendo en cuenta lo ya visto, para analizar y diseñar circuitos combinacionales sólo queda por ver los tipos de circuitos integrados digitales que hay de éste tipo y qué funciones realizan.

Para poder analizar y estudiar los circuitos integrados comerciales con un cierto orden, es necesario clasificarlos de acuerdo con algún criterio. A continuación se muestra una clasificación de los circuitos integrados en función del número de puertas que contienen.

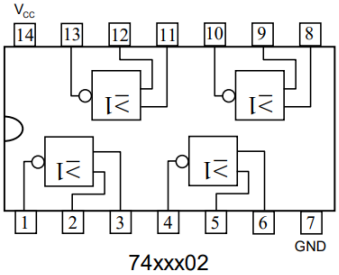
Escala de integración	Nº de puertas/IC	Aplicación
SSI (Small scale integration)	1 – 10	Puertas lógicas y biestables
MSI (Medium scale integration)	10 – 100	Decodificadores, codificadores, multiplexores, circuitos aritméticos, contadores, registros, etc.
LSI (Large scale integration)	100 – 10^3	Pequeños microprocesadores, memorias, PLDs, etc.
VLSI (Very large scale integration)	10^3 – 10^4	Microprocesadores, memorias, PLDs, etc.
ULSI (Ultra large scale integration)	10^4 – 10^5	Microprocesadores, memorias, PLDs, FPGAs, DSPs, Microcontroladores.
GLSI (Giga large scale integration)	> 10^5	Microprocesadores

Curiosidad: el número de transistores que había en los chips han ido variando en número desde hace años. A principios de los años 80 del siglo pasado se fabricaban chips en cuyo interior había $0{,}1\times10^6$ transistores; mientras que en 2009 se fabricaban chips en cuyo interior había 1000×106 transistores. Al final siempre va a existir un límite físico al número de transistores que se pueden meter en un chip.

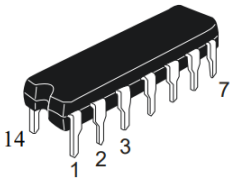
Ejemplos de circuitos integrado de la escala de integración pequeña (SSI):

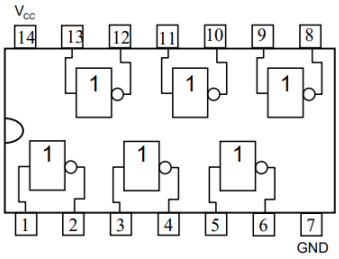


74xxx00

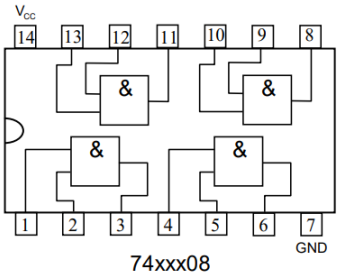


74xxx02





74xxx04



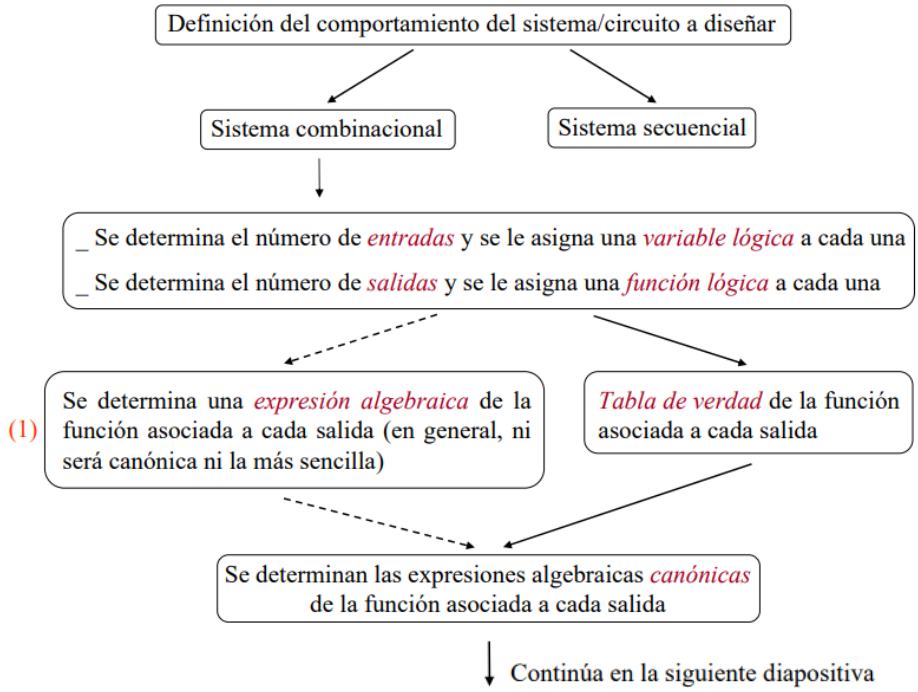
74xxx08

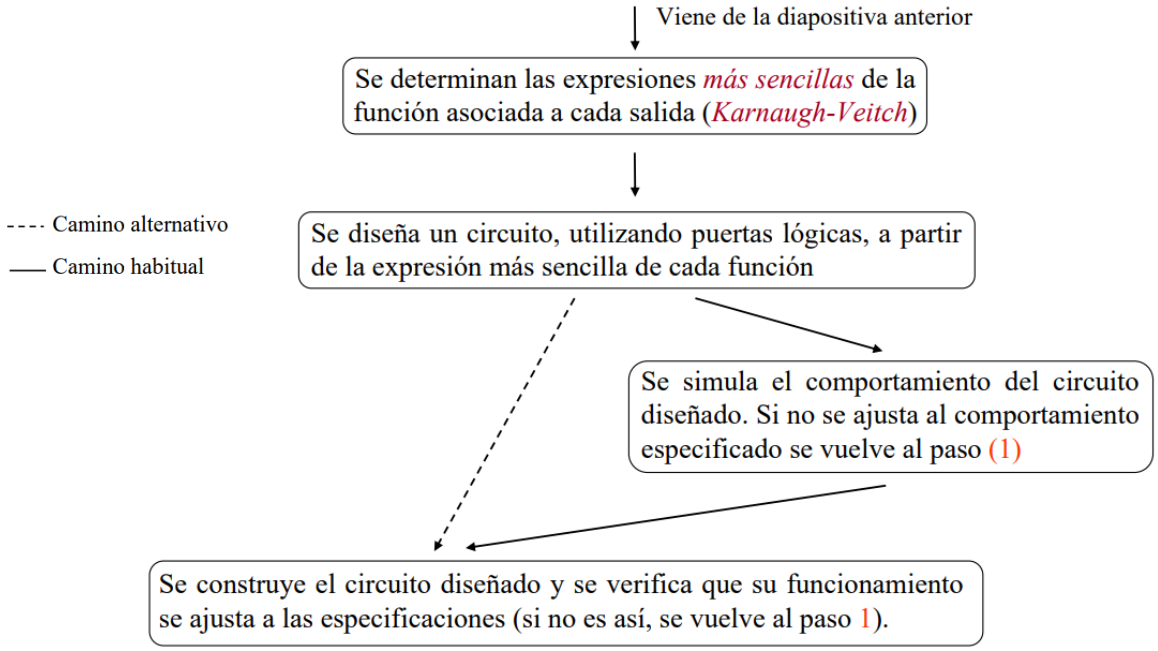
Diseño (síntesis) de circuitos combinacionales sencillos utilizando circuitos integrados SSI.

En este tema sólo se va a ver cómo se analizan y diseñan sistemas combinacionales utilizando circuitos integrados de la escala de integración baja (SSI).

SSI ⇔ puertas lógicas

El proceso de diseño(síntesis) de un sistema combinacional utilizando puertas lógicas(circuitos integrados de la escala SSI) se puede esquematizar de la siguiente manera:





El procedimiento de diseño anterior se puede indicar de la siguiente manera:

1º paso: A partir de la descripción del comportamiento del circuito a diseñar se determina si dicho circuito corresponde a un sistema combinacional o secuencial. En el caso de que sea un sistema secuencial hay que esperar a ver el tema 5 y en el caso de que sea un sistema combinacional, se continúa con el siguiente paso (2).

2º paso: Se determina el número de entradas y se le asigna una variable lógica a cada una. Se determina el número de salidas y se le asigna una función lógica a cada una.

3º paso: A partir de la descripción del comportamiento del circuito a diseñar se determina una expresión lógica de cada función a implementar o bien se determina su tabla de verdad.

4º paso: A partir de las expresiones lógicas o de las tablas de verdad de las distintas funciones obtenidas en el paso anterior se determinan sus expresiones canónicas.

5º paso: Se determinan las expresiones más sencillas de las funciones a implementar utilizando el método de Karnaugh-Veitch.

6º paso: A partir de la expresión más sencilla de cada función se diseña un circuito que las implemente, utilizando puertas lógicas.

7º paso (opcional): Se simula el comportamiento del circuito diseñado en el paso anterior. Si el comportamiento del circuito no se ajusta a lo especificado inicialmente se vuelve al paso 3.

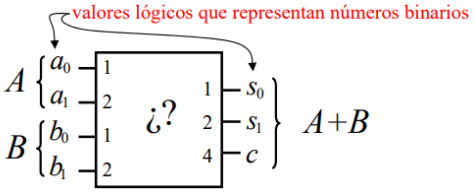
8º paso: Se construye el circuito y se comprueba que cumple las especificaciones (en el caso de que no sea así, se vuelve al paso 3).

Ejemplo: Diseño de un circuito que sume dos números binarios A(a_1, a_0) y B(b_1, b_0), sin signo, utilizando puertas lógicas.

1º paso: El circuito a diseñar corresponde a un sistema combinacional.

2º paso: Se determina el número de entradas (a_1, a_0, b_1, b_0) y de salidas (c, s_1, s_0) del circuito a diseñar.

$$\begin{array}{rccccl} & 4 & 2 & 1 & & \\ & & a_1 & a_0 & & \equiv A \\ + & & b_1 & b_0 & & \equiv B \\ \hline & c & s_1 & s_0 & & \equiv A+B \end{array}$$



3º paso: en la siguiente página se indica la tabla de verdad de las funciones a implementar.

Nota: Los valores de los números binarios (a1 a0)2 y (b1 b0)2 se representan a nivel lógico mediante 1s y 0s lógicos. Es decir, se realiza la siguiente analogía:

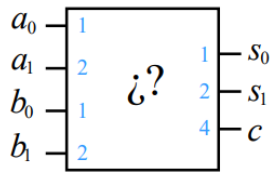
1 *aritmético* ≡ 1 *lógico* ≡ 5V 0 *aritmético* ≡ 0 *lógico* ≡ 0V

$$s_0(a_1,a_0,b_1,b_0)=\sum_4(1,3,4,6,9,11,12,14)=\prod_4(0,2,5,7,8,10,13,15)$$

$$s_1(a_1,a_0,b_1,b_0)=\sum_4(2,3,5,6,8,9,12,15)=\prod_4(1,2,4,5,8,11,14,15)$$

$$c(a_1,a_0,b_1,b_0)=\sum_4(7,10,11,13,14,15)=\prod_4(3,6,7,9,10,11,12,13,14,15)$$

2	1	2	1	4	2	1
a_1	a_0					
0	0					
0	0					
0	0					
0	0	1	1	0	1	1
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	1
0	1	1	1	1	0	0
1	0	0	0	0	1	0
1	0	0	1	0	1	1
1	0	1	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	0	1	1	0	0
1	1	1	0	1	0	1
1	1	1	1	1	1	0



$$A=1,\quad B=2,\quad A+B=3$$

$$\left.\begin{array}{l} A\equiv a_1a_0=01_2 \\ B\equiv b_1b_0=10_2 \end{array}\right\}\rightarrow cs_1s_0=011_2$$

valor lógico que representa un 1_2 (aritmético)

19

4º paso: Se determinan las expresiones canónicas de las funciones a implementar:

5º paso: Se determinan las expresiones más sencillas de las funciones a implementar utilizando el método de Karnaugh-Veitch:

$$s_0(a_1,a_0,b_1,b_0)=\sum\nolimits_4(1,3,4,6,9,11,12,14)=\overline{a}_0b_0+a_0\overline{b}_0=a_0\oplus b_0$$

$$s_1(a_1,a_0,b_1,b_0)=\sum\nolimits_4(2,3,5,6,8,9,12,15)=\overline{a}_1\overline{a}_0b_1+\overline{a}_1b_1\overline{b}_0+\overline{a}_1a_0\overline{b}_1b_0+a_1a_0b_1b_0+$$
$$+a_1\overline{b}_1\overline{b}_0+a_1\overline{a}_0\overline{b}_1=\cdots=a_1\oplus b_1\oplus(a_0b_0)$$

$$c(a_1,a_0,b_1,b_0)=\sum\nolimits_4(7,10,11,13,14,15)=a_1a_0b_0+a_0b_1b_0+a_1b_1=(a_1+b_1)a_0b_0+a_1b_1$$

Comentario: la tabla de Karnaugh-Veitch para simplificar la función. presenta una distribución de 1s con una simetría, en este caso, impar. Esto implica que al menos parte de la expresión más sencilla de s1 se puede expresar en función de sumas xor y/o xnor.

a_1a_0 \ b_1b_0	00	01	11	10
00	0	1	1 (3)	1 (2)
01	4	1 (5)	7	1 (6)
11	1 (12)	13	1 (15)	14
10	1 (8)	1 (9)	11	10

$$s_1(a_1,a_0,b_1,b_0)=\sum\nolimits_4(2,3,5,6,8,9,12,15)$$

Demostración expresión de s1: a partir de la expresión más sencilla se s1 obtenida utilizando el método de Karnaugh-Veitch:

$$s_1(a_1,a_0,b_1,b_0)=\sum\nolimits_4(2,3,5,6,8,9,12,15)=$$

$$=\overline{a}_1\overline{a}_0b_1+\overline{a}_1b_1\overline{b}_0+\overline{a}_1a_0\overline{b}_1b_0+a_1a_0b_1b_0+a_1\overline{a}_0\overline{b}_1+a_1\overline{b}_1\overline{b}_0$$

Se deduce lo siguiente:

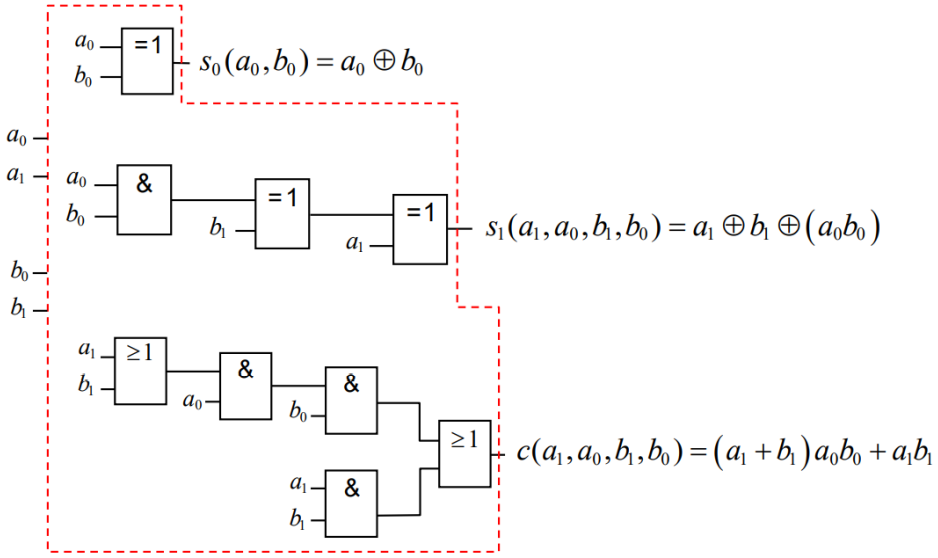
$$s_1(a_1,a_0,b_1,b_0)=\overline{a}_1\overline{a}_0b_1+\overline{a}_1b_1\overline{b}_0+\overline{a}_1a_0\overline{b}_1b_0+a_1a_0b_1b_0+a_1\overline{a}_0\overline{b}_1+a_1\overline{b}_1\overline{b}_0=$$

$$=(\overline{a}_1\overline{b}_1+a_1b_1)a_0b_0+a_1\overline{b}_1(\overline{a}_0+\overline{b}_0)+\overline{a}_1b_1(\overline{a}_0+\overline{b}_0)=$$

$$=(\overline{a_1\oplus b_1})a_0b_0+(a_1\overline{b}_1+\overline{a}_1b_1)(\overline{a}_0+\overline{b}_0)=$$

$$=\overline{a_1\oplus b_1}\;a_0b_0+(a_1\oplus b_1)\overline{a_0b_0}=a_1\oplus b_1\oplus(a_0b_0)$$

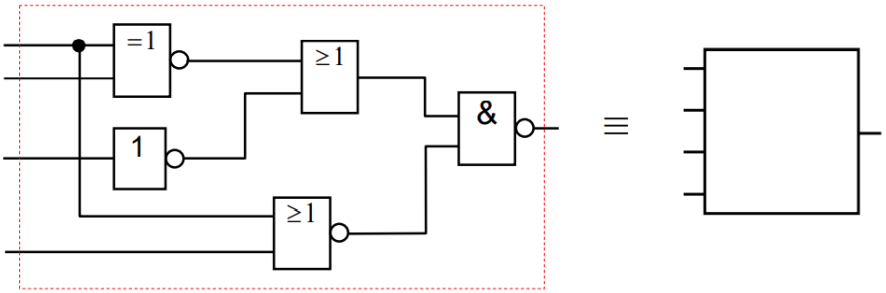
6º paso: Se dibuja un esquema de los circuitos que implementan las funciones c, s1 y s0 a partir de las expresiones simplificadas (la implementación no es única).



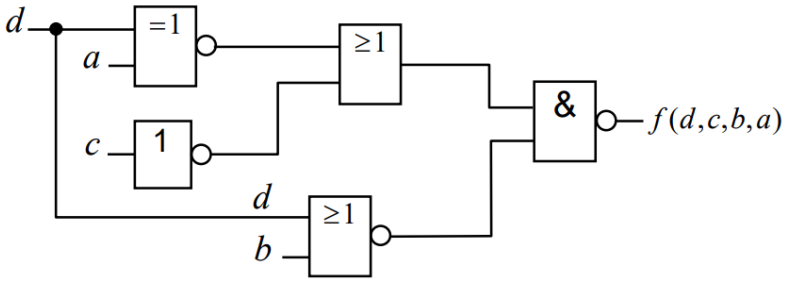
Análisis de sistemas combinacionales realizados con circuitos integrados SSI.

"Conocemos el esquema de un circuito combinacional y queremos determinar qué hace dicho circuito ≡ qué función o funciones lógicas implementa".

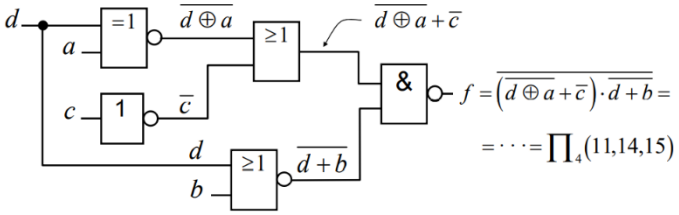
Ejemplo:



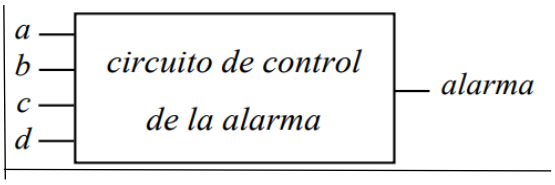
1º paso: Se asignan variables a las entradas y funciones a las salidas.



2º paso: Partiendo de las entradas, se van determinando las expresiones lógicas correspondientes a las salidas de las distintas puertas lógicas hasta determinar la expresión lógica de la salida (o salidas) del circuito.



Ejercicio: Al circuito de control de la alarma contra robos del Banco de Sereno llegan cuatro hilos conductores:



- el conductor a procede de un interruptor secreto.
- el conductor b procede de un sensor de presión situado debajo de la caja fuerte.
- el conductor c procede de un reloj electrónico alimentado por baterías.
- el conductor d procede del sistema electromecánico de apertura de la caja fuerte.

Las siguientes condiciones provocan una tensión de 5 voltios (con respecto a masa) en el correspondiente conductor:

- V_a = 5V si el interruptor secreto esta pulsado.
- V_b = 5V si cambia el peso de la caja fuerte.
- V_c = 5V si el reloj marca entre las $10{:}00_{AM}$ y las $14{:}00_{PM}$.
- V_d = 5V si se introduce un código de apertura incorrecto a través del teclado alfanumérico situado en la puerta de la caja fuerte.

La lógica de control de la alarma contra robos ha sido programada dela siguiente forma:

a) Suena la alarma entre las $10{:}00_{AM}$ y las $14{:}00_{PM}$ en los siguientes casos:
   a. Si están pulsando el interruptor secreto, cambie el peso de la caja fuerte.
   b. Si se introduce un código de apertura de la caja fuerte incorrecto.
b) Suena la alarma entre las $14{:}00_{PM}$ de un día y las $10{:}00_{AM}$ del día siguiente en los siguientes casos:
   a. Si se introduce un código de apertura incorrecto y no esta pulsado el interruptor secreto.
   b. Si cambia el peso de la caja fuerte.

Obtener una función lógica que describa el comportamiento de la alarma del banco.

Acuerdo: $$a = \begin{cases} 1 & \text{si el interruptor secreto está pulsado} \\ 0 & \text{si el interruptor secreto no está pulsado} \end{cases}$$

$$b = \begin{cases} 1 & \text{si cambia el peso de la caja fuerte} \\ 0 & \text{si no cambia el peso de la caja fuerte} \end{cases}$$

$$c=\begin{cases}1 & \text{si es entre las } 10{:}00_{AM} \text{ y las } 14{:}00_{PM}\\ 0 & \text{si es entre las } 14{:}00_{PM} \text{ y las } 10{:}00_{AM}\end{cases}$$

$$d=\begin{cases}1 & \text{si se introduce un código de apertura incorrecto}\\ 0 & \text{si se introduce un código de apertura correcto}\end{cases}$$

$$alarma(d,c,b,a)=\begin{cases}1 & \text{si suena la alarma del banco}\\ 0 & \text{si no tiene que sonar la alarma}\end{cases}$$

Solución: $alarma(d,c,b,a)=c(a\cdot b+d)+\overline{c}(d\cdot\overline{a}+b)$

Ejercicio: Una planta química utiliza un tanque de fibra de vidrio para almacenar un determinado compuesto altamente inflamable. Para supervisar en todo momento el estado de dicho compuesto, se han conectado tres sensores al tanque:

- El sensor **z** indica, mediante una variable binaria, cuando la temperatura T en el interior del tanque supera un valor T_1 dado.
- El sensor **y** indica, mediante una variable binaria, cuando la temperatura T en el interior del tanque supera un valor T_2 dado. Siendo $T_2 > T_1$.
- El sensor **x** indica, mediante una variable binaria, cuando la presión P en el interior del tanque supera un valor P_1 dado.

a) Determinar la tabla de verdad de la función lógica que representa las condiciones óptimas de almacenamiento del compuesto, definidas por: $T_1 < T < T_2$ y $P < P_1$.

b) Determinar las funciones canónicas de la función anterior.

Nota: supóngase que los sensores no se pueden averiar.

Acuerdo:

$$z=\begin{cases}1 & \text{si } T>T_1\\ 0 & \text{en caso contrario}\end{cases}\qquad y=\begin{cases}1 & \text{si } T>T_2>T_1\\ 0 & \text{en caso contrario}\end{cases}\qquad x=\begin{cases}1 & \text{si } P>P_1\\ 0 & \text{en caso contrario}\end{cases}$$

$$f(z,y,x)=\begin{cases}1 & \text{si el compuesto está almacenado en las } \textit{condiciones óptimas}\\ 0 & \text{si el compuesto no está almacenado en las } \textit{condiciones óptimas}\end{cases}$$

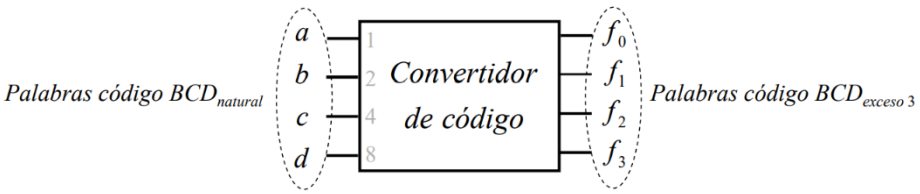
$$\textit{Condiciones óptimas}:\begin{cases}T_1<T<T_2 & \Rightarrow\ z=1\,,\ y=0\\ P<P_1 & \Rightarrow\ x=0\end{cases}$$

Solución: $f(z,y,x)=\sum_3(4)+\sum_\phi(2,3)$

$$f(z,y,x)=\prod_3(0,1,2,6,7)\cdot\prod_\phi(4,5)$$

z	y	x	f
0	0	0	0
0	0	1	0
0	1	0	x
0	1	1	x
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Ejercicio: Obtener las expresiones algebraicas canónicas de un circuito convertidor de palabras código $BCD_{natural}$ a palabras código $BCD_{exceso\ 3}$.



Solución:

$$f_0(d,c,b,a) = \sum_4 (0,2,4,6,8) + \sum_\phi (10,11,12,13,14,15)$$

$$f_1(d,c,b,a) = \sum_4 (0,3,4,7,8) + \sum_\phi (10,11,12,13,14,15)$$

$$f_2(d,c,b,a) = \sum_4 (1,2,3,4,9) + \sum_\phi (10,11,12,13,14,15)$$

$$f_3(d,c,b,a) = \sum_4 (5,6,7,8,9) + \sum_\phi (10,11,12,13,14,15)$$

	$BCD_{natural}$				$BCD_{exceso\text{-}3}$			
	d	c	b	a	f_3	f_2	f_1	f_0
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0
	1	0	1	0	x	x	x	x
	1	0	1	1	x	x	x	x
	1	1	0	0	x	x	x	x
	1	1	0	1	x	x	x	x
	1	1	1	0	x	x	x	x
	1	1	1	1	x	x	x	x

Ejercicio: El avión de combate, modelo CHAPUZA 16, incorpora el circuito X47JK como último avance tecnológico. La función de este circuito es informar al piloto del estado del tren de aterrizaje del avión mediante indicadores luminosos.

Al circuito X47JK llegan los conductores a, b y c, procedentes de los circuitos que supervisan la posición de los trenes trasero izquierdo, trasero derecho y delantero respectivamente. Estos conductores presentan un nivel de tensión alto siempre que el correspondiente tren esté bajado, y un nivel de tensión bajo siempre que el correspondiente tren esté subido.

El circuito X47JK activa dos indicadores luminosos (L1 y L2) de color verde, un indicador luminoso (L3) de color rojo y un indicador luminoso (L4) de color azul. El indicador L1 se enciende siempre que todo el tren de aterrizaje está correctamente subido; el indicador L2 se enciende siempre que todo el tren de aterrizaje está correctamente bajado; el indicador L3 se enciende siempre que uno de los trenes de aterrizaje está averiado y el indicador L4 se enciende siempre que el tren de aterrizaje delantero está averiado. Determinar las funciones lógicas que gobiernan el estado de los indicadores luminosos L1, L2, L3 y L4.

Nota: Supóngase nula la probabilidad de que estén averiados simultáneamente dos o más trenes de aterrizaje.

Solución: asistir a la corrección de la tarea 7.

Inicio del Tema 4

Introducción a los bloques funcionales combinacionales(comerciales)

Escala SSI (hasta 10 puertas/chip) $\Leftrightarrow$ funciones sencillas (pocas).

Escala MSI (hasta 100 puertas/chip) $\Rightarrow$ realización de una o varias funciones (complejas) de aplicación general.

Ventajas de la utilización de ICs MSI:

a) Se reduce el número de ICs a utilizar.
b) Se reduce el coste del circuito.
c) Se simplifica el diseño (teórico).
d) Se simplifica el cableado del circuito y su construcción.
e) Lograr que el circuito no presente fenómenos aleatorios y problemas de EMC (compatibilidad electromagnética) resulta más fácil.

La filosofía de diseño cambió con la aparición de los ICs de la escala MSI:

a) Objetivo principal: minimizar el número de ICs a utilizar.
b) Objetivo secundario: minimizar el número de puertas lógicas a utilizar.

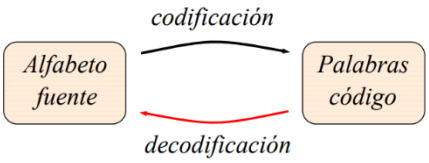
En este tema se estudian los principales bloques funcionales combinacionales pertenecientes a la escala de integración media (MSI). Dichos bloques se utilizan habitualmente en la descripción de la arquitectura interna de circuitos digitales complejos como son, por ejemplo, procesadores, microcontroladores, DSPs-DSCs, etc.

Comentario: los circuitos digitales no se diseñan siguiendo la filosofía que aquí se explica desde hace años… la existencia de los lenguajes de descripción de hardware, de las FPGAs y de las CPLDs ha cambiado el procedimiento de diseño e implementación de los circuitos digitales. Ahora bien, el conocimiento de los bloques funcionales que se estudian en este tema y, en general, en esta asignatura son esenciales para poder diseñar circuitos digitales y para poder entender la arquitectura interna de procesadores, µC, DSPs y de ASICs.

Decodificadores de n líneas de entrada a m líneas de salida, de 1 línea de salida activa entre m líneas de salida.

Def: Un decodificador es un circuito combinacional que decodifica información codificada en un código dado…. decodifica palabras código.

Un decodificador X/Y indica para cada palabra código de un código X el símbolo del código fuente Y que le corresponde.



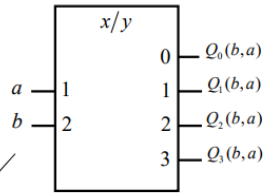
Ejemplo:

Alfabeto Fuente	Palabras código $BCD_{natural}$	Alfabeto código
0	0 0 0 0	0
1	0 0 0 1	1
2	0 0 1 0	
3	0 0 1 1	
4	0 1 0 0	
5	0 1 0 1	
6	0 1 1 0	
7	0 1 1 1	
8	1 0 0 0	
9	1 0 0 1	

Def 2: Un decodificador es un circuito combinacional con n entradas y m salidas. Para cada combinación (válida) de las n variables de entrada sólo se activa una de las m salidas. Dicho de otro modo, permite seleccionar la salida que está activa.

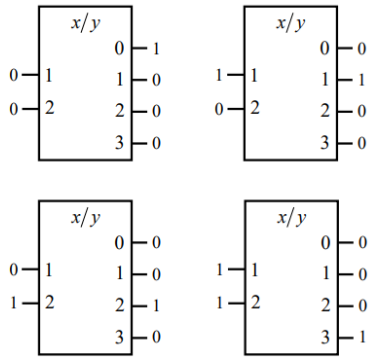
Notas: todos los decodificadores que se estudian en esta asignatura cumplen la definición anterior. Ahora bien, existen decodificadores, que no se estudian en esta asignatura por falta de tiempo, que no cumplen la definición anterior como, por ejemplo, un decodificador de BCD a 7 segmentos.

Decodificador de 2 a 4 ó de 1 entre 4:

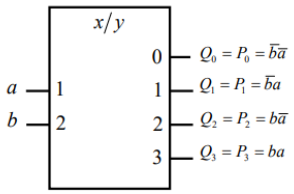


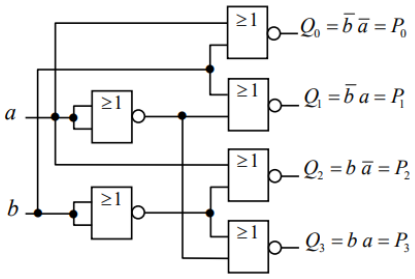
2 b	1 a	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Las entradas tienen asignados pesos (la entrada b tiene asignado el peso 2 y la entrada a tiene asignado el peso 1). Las salidas están numeradas empezando desde 0. Se pone a 1 la salida que tiene asignado un número igual a la suma de los pesos de las entradas que están a 1.

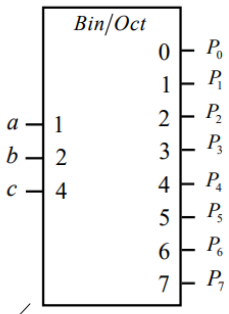


Implementación de un decodificador de 2 a 4 utilizando puertas NOR de 2 entradas:





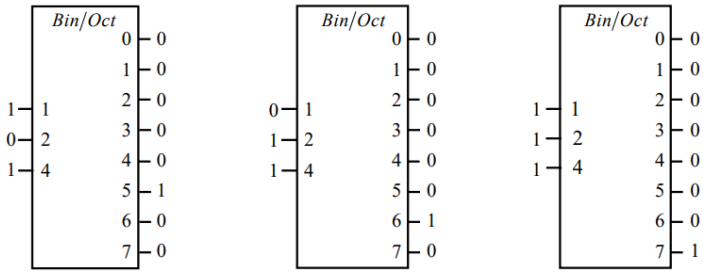
Decodificador de 3 a 8 o de 1 entre 8 ≡ Decodificador octal:



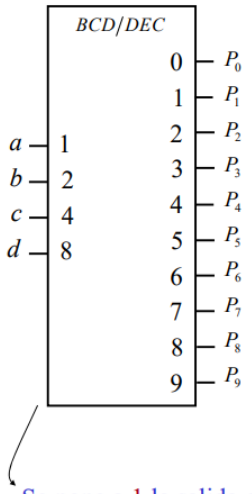
4 c	2 b	1 a	P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

Se pone a 1 la salida que tiene asignado un número igual a la suma de los pesos de las entradas que están a 1. Los pesos asociados a las entradas c, b y a son 4, 2 y 1 respectivamente.

Ejemplos:



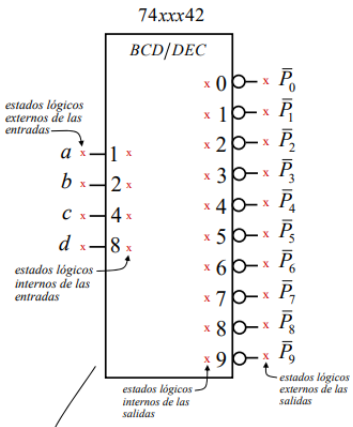
Decodificador Decimal:



8	4	2	1										
d	c	b	a	P_9	P_8	P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0
0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	1	0	0	0	0
0	1	0	1	0	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	0	1	0	0	0	0	0	0
0	1	1	1	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	1	1	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0	0	0	0

Se pone a 1 la salida que tiene asignado un número igual a la suma de los pesos de las entradas que están a 1. Los pesos asociados a las entradas *d*, *c*, *b* y *a* son 8, 4, 2 y 1 respectivamente. El mayor valor que se puede decodificar es el 1001_2

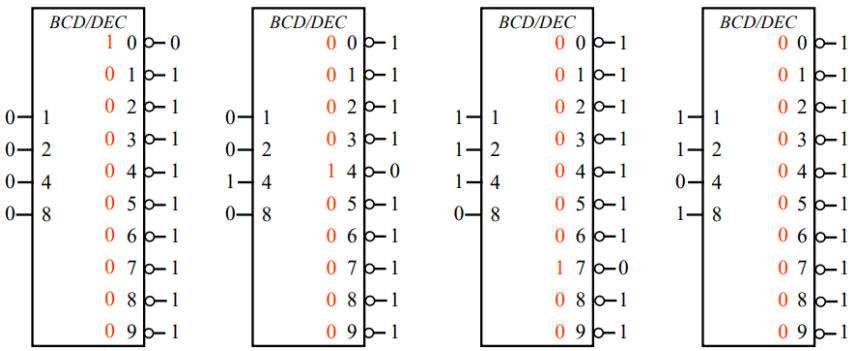
Decodificador Decimal Negado:



d	c	b	a	$\bar{P}_9$	$\bar{P}_8$	$\bar{P}_7$	$\bar{P}_6$	$\bar{P}_5$	$\bar{P}_4$	$\bar{P}_3$	$\bar{P}_2$	$\bar{P}_1$	$\bar{P}_0$
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	1	1	1	1	1	1	1	1	1	0	1
0	0	1	0	1	1	1	1	1	1	1	0	1	1
0	0	1	1	1	1	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	1	1	0	1	1	1	1
0	1	0	1	1	1	1	1	0	1	1	1	1	1
0	1	1	0	1	1	1	0	1	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1	1	1	1
1	0	0	0	1	0	1	1	1	1	1	1	1	1
1	0	0	1	0	1	1	1	1	1	1	1	1	1
1	0	1	0	1	1	1	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1	1	1	1	1	1	1
1	1	0	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

Se pone a 1 el estado lógico interno de la salida que tiene asignado un número igual a la suma de los pesos de las entradas que están a 1. Los pesos asociados a las entradas *d*, *c*, *b* y *a* son 8, 4, 2 y 1 respectivamente. El mayor valor que se puede decodificar es el 1001_2

Ejemplos:

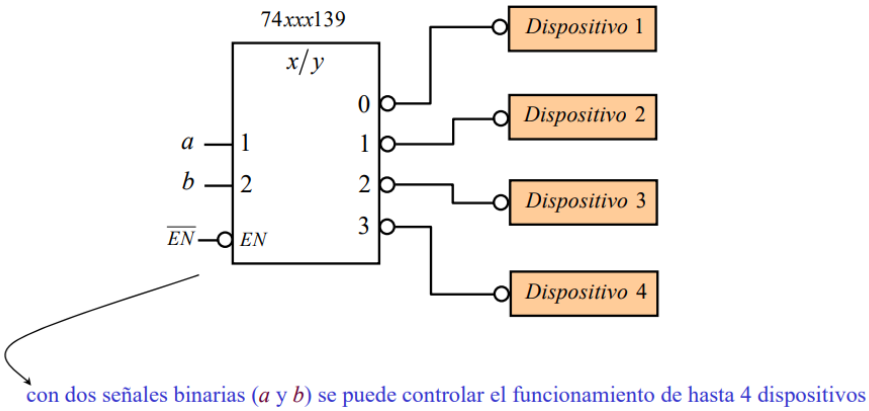


Aplicaciones:

1ª: Decodificar palabras código(ejemplos anteriores).

2ª: Control de funcionamiento no simultaneo de varios dispositivos.

Ejemplo:



3ª: Construir decodificadores de otros tipos de códigos : por ejemplo, utilizando decodificadores :

a) Un decodificador de cualquier código BCD (Aiken, exceso-3, etc).
b) Un Decodificar hexadecimal.
c) etc.

4ª: Implementar funciones lógicas (en algunos casos se consigue reducir el número de ICs a utilizar).

Ejemplo 1: Diseñar un circuito que implemente las siguientes funciones, utilizando un decodificador adecuado y puertas NAND.

$$f_1(c,b,a) = \sum_3 (0,1,3,5)$$

$$f_2(c,b,a) = \sum_3 (5,6,7) + \sum_\phi (1,4)$$

Ejemplo 2: Diseñar un circuito que implemente la siguiente función, utilizando un decodificador decimal y puertas NAND.

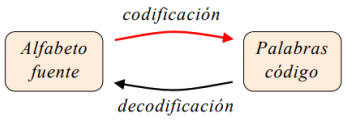
$$g(b,a) = \sum_2 (0,1,2)$$

Ejemplo 3: Diseñar un circuito que implemente la siguiente función, utilizando un decodificador adecuado y el menor número posible de puertas NAND de 2 entradas.

$$h(d,c,b,a) = \overline{\overline{\overline{d} \cdot c \cdot \overline{a}} \cdot (c + b + \overline{a})}$$

Codificadores de n líneas de entrada a m líneas de salida.

Def: Un codificador es un circuito combinacional que proporciona para cada símbolo de un alfabeto fuente dado la palabra código que le corresponde según el código que se considere.



Hay dos tipos de codificadores: con prioridad y sin prioridad. Su funcionamiento es el mismo excepto en el caso de que se activen simultáneamente dos o más entradas. Los codificadores sin prioridad no se utilizan.

Ejemplo:

Alfabeto Fuente	Palabras código $BCD_{natural}$	Alfabeto código
0	0 0 0 0	0
1	0 0 0 1	1
2	0 0 1 0	
3	0 0 1 1	
4	0 1 0 0	
5	0 1 0 1	
6	0 1 1 0	
7	0 1 1 1	
8	1 0 0 0	
9	1 0 0 1	

Los codificadores que se estudian en esta asignatura proporcionan la palabra código correspondiente a la entrada activa que tiene el mayor valor asociado (codificadores con prioridad).

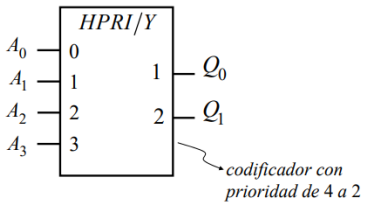
Nota: un codificador realiza la función inversa de la que realiza un decodificador. El valor presente en sus salidas indica la entrada que activa que tiene asignado un valor mayor.

Codificadores con prioridad (o de alta prioridad ≡ high priority): cuando se activan simultáneamente dos o más entradas, en la salida aparece la codificación correspondiente a la entrada activa que tenga asignado el valor más alto. Los codificadores de alta prioridad se utilizan para controlar el acceso a recursos compartidos. Si dos o más peticiones son recibidas el codificador indica cual de las peticiones tiene una mayor prioridad.

Curiosidad: una de las aplicaciones de los codificadores con prioridad es la de gestionar el acceso a recursos compartidos. Si se reciben dos o más peticiones de uso de un mismo recurso, es un codificador con prioridad el que decide cuál de las solicitudes tiene mayor prioridad y, por lo tanto, accede primero al recurso solicitado.

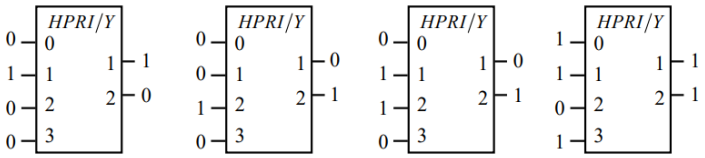
En microcontroladores avanzados, en DSPs y en DSCs cuando se activa el flag de dos o más fuentes de interrupción, se utiliza un codificador con prioridad para determinar la fuente de interrupción que la CPU atiende en primer lugar.

Codificadores con prioridad (o de alta prioridad ≡ high priority): cuando se activan simultáneamente dos o más entradas, en la salida aparece la codificación (palabra código) correspondiente a la entrada activa que tenga asignado el valor más alto.



A_3	A_2	A_1	A_0	Q_1	Q_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1
	⋮			⋮	
0	1	1	0	1	0
	⋮			⋮	
0	0	0	0	0	0

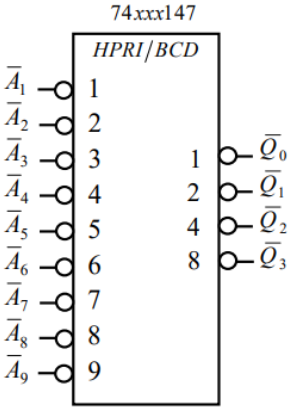
Ejemplos:



Problema: observando las salidas no se puede distinguir entre la situación en la que sólo está activa la entrada que tiene asignado el valor 0 y la situación en la que no hay ninguna entrada activa. (Imágenes encima de ejemplos)

Solución: Omitir la entrada de menor peso/prioridad.

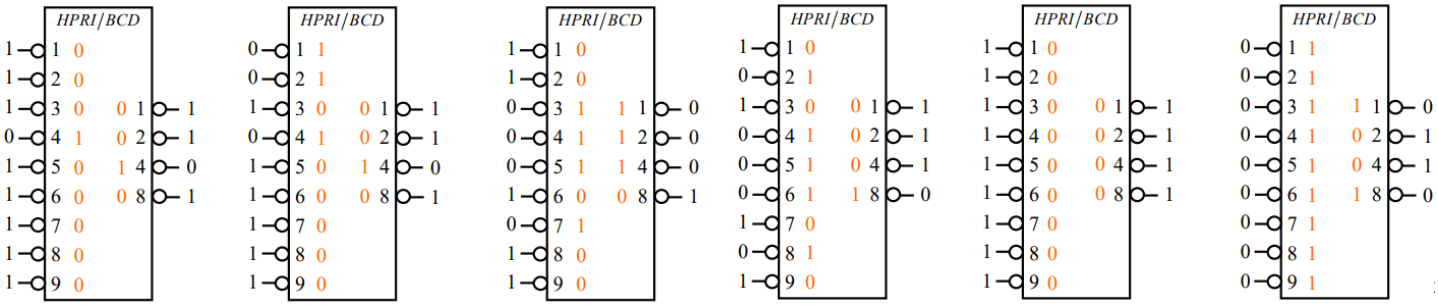
Ejemplo: Codificador con prioridad de Decimal a $BCD_{natural}$ ó de 9 a 4.



$\overline{A}_9$	$\overline{A}_8$	$\overline{A}_7$	$\overline{A}_6$	$\overline{A}_5$	$\overline{A}_4$	$\overline{A}_3$	$\overline{A}_2$	$\overline{A}_1$	$\overline{Q}_3$	$\overline{Q}_2$	$\overline{Q}_1$	$\overline{Q}_0$
0	x	x	x	x	x	x	x	x	0	1	1	0
1	0	x	x	x	x	x	x	x	0	1	1	1
1	1	0	x	x	x	x	x	x	1	0	0	0
1	1	1	0	x	x	x	x	x	1	0	0	1
1	1	1	1	0	x	x	x	x	1	0	1	0
1	1	1	1	1	0	x	x	x	1	0	1	1
1	1	1	1	1	1	0	x	x	1	1	0	0
1	1	1	1	1	1	1	0	x	1	1	0	1
1	1	1	1	1	1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1

Las entradas están numeradas consecutivamente, en este caso, empezando en 1. Las salidas tienen asignados pesos. Los estados lógicos internos de las salidas toman, de acuerdo con los pesos que tienen asignados, el valor correspondiente al número más alto de todas las entradas que estén activas (entradas cuyo estado lógico interno sea igual a 1).

Ejemplos:

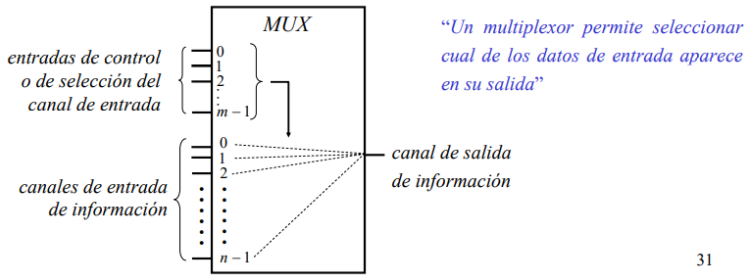


Multiplexores de n canales (de entrada) o selector de datos.

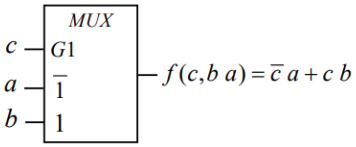
Def: Un multiplexor es un circuito integrado que tiene:

a) salida (canal) de información.
b) m entradas de control o de selección (de uno de los canales de entrada).
c) n entradas (canales) de información, cumpliéndose que: 2m = n.

Su funcionamiento se caracteriza por lo siguiente: "La combinación binaria aplicada a las m entradas de control selecciona el canal de entrada cuya información lógica (1 ó 0) aparece en el canal de salida".



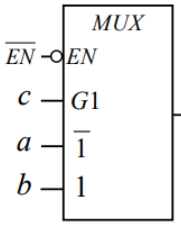
Multiplexores de 2 canales:



$f(c,b\,a) = \overline{c}\,a + c\,b$

c	f
0	a
1	b

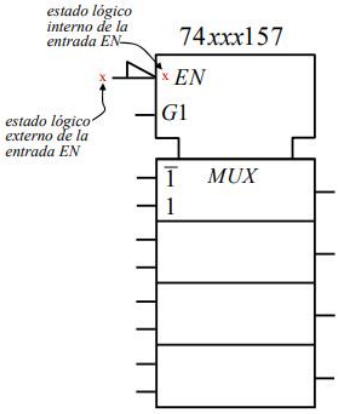
→ f vale 1 si c vale 0 y a vale 1 o bien si c vale 1 y b vale 1



$f(\overline{EN},c,b,a) = \overline{\overline{EN}}\cdot\overline{c}\cdot a + \overline{\overline{EN}}\cdot c\cdot b = EN(\overline{c}\cdot a + c\cdot b)$

$\overline{EN}$	c	b	a	f
1	x	x	x	0
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1

Cuádruple multiplexor de 2 canales: los cuatro multiplexores comparten la entrada de selección y la entrada de inhibición (En: enable).

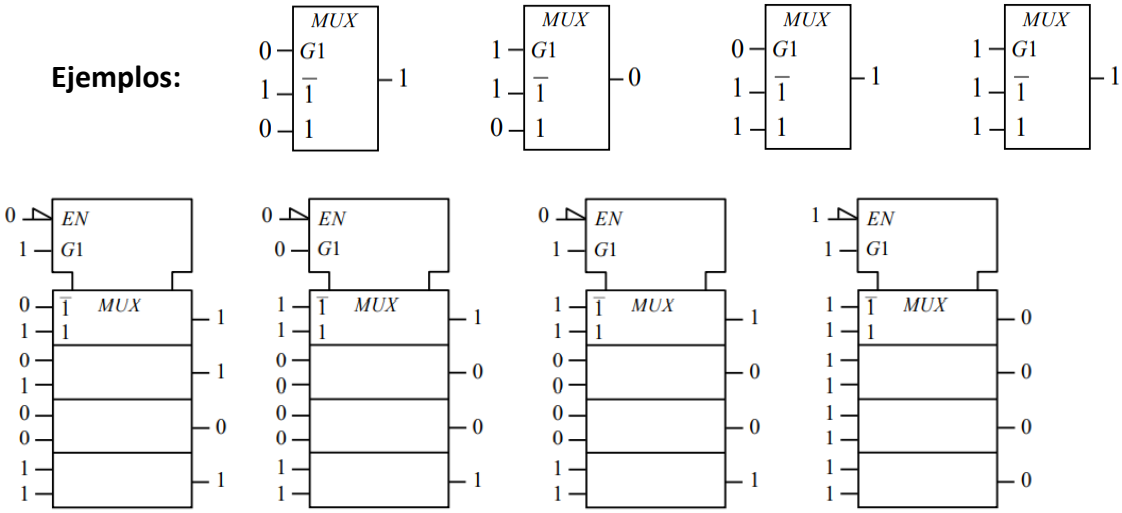


Nota: cuando en una entrada de habilitación, después de *EN* no hay nada (un número o una letra), y las salidas no se pueden poner en *tercer estado* (no tienen el símbolo ▽), se cumple lo siguiente:

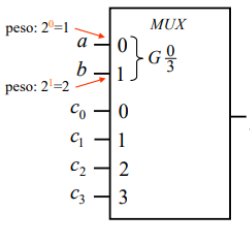
· Si el *estado lógico interno* de la entrada *EN* es igual a 1, entonces el circuito funciona normalmente.

· Si el *estado lógico interno* de la entrada *EN* es igual a 0, entonces los *estados lógicos internos* de todas las salidas del circuito se ponen a 0, con independencia de los valores que tomen las demás entradas.

Ejemplos:



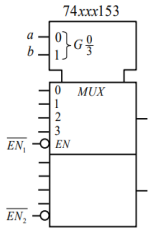
Multiplexor de 4 canales:



$$f(c_3,c_2,c_1,c_0,b,a) = c_0\overline{b}\,\overline{a} + c_1\overline{b}\,a + c_2 b\,\overline{a} + c_3 b\,a$$

b	a	f
0	0	c_0
0	1	c_1
1	0	c_2
1	1	c_3

Doble multiplexor de 4 canales:

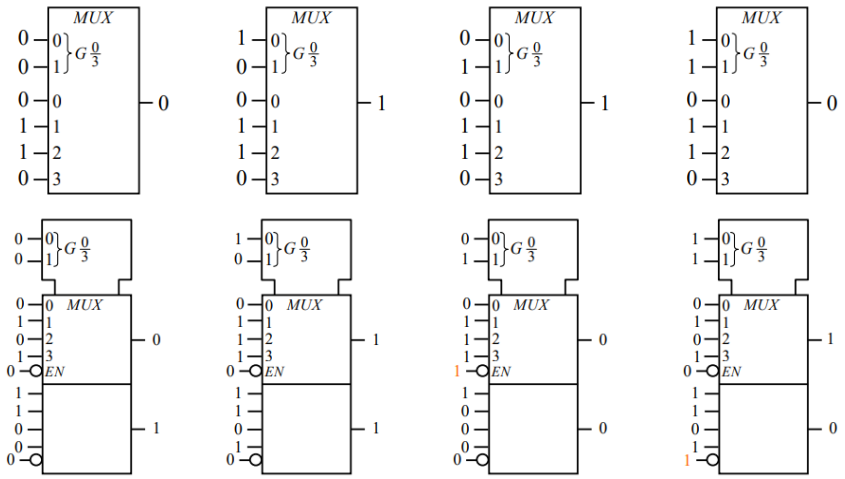


Nota: en este circuito se cumple lo siguiente:

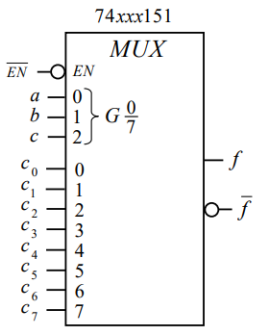
_ Los multiplexores se habilitan/deshabilitan de forma independiente

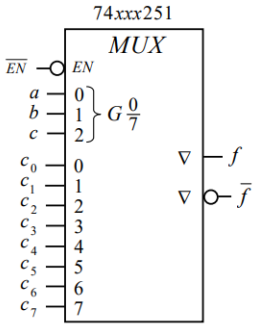
_ Ambos multiplexores comparten las entradas de selección (*b* y *a*)

Ejemplos:



Multiplexores de 8 canales:



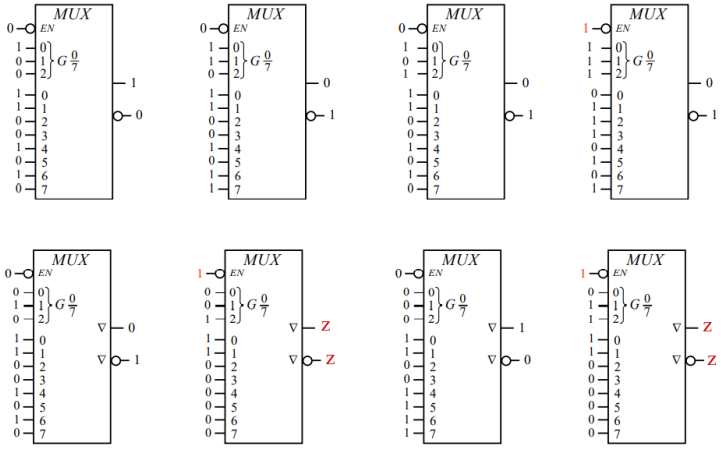


Nota: cuando una salida lleva el símbolo ∇ significa que dicha salida puede ponerse en un *tercer estado* (≡ estado de muy alta impedancia), además de poder ponerse en los dos estados normales: 1 y 0.

Nota: cuando después del símbolo calificador EN no hay ni un número ni una letra (no hay nada) y las salidas pueden ponerse en un tercer estado ( ), se cumple lo siguiente:

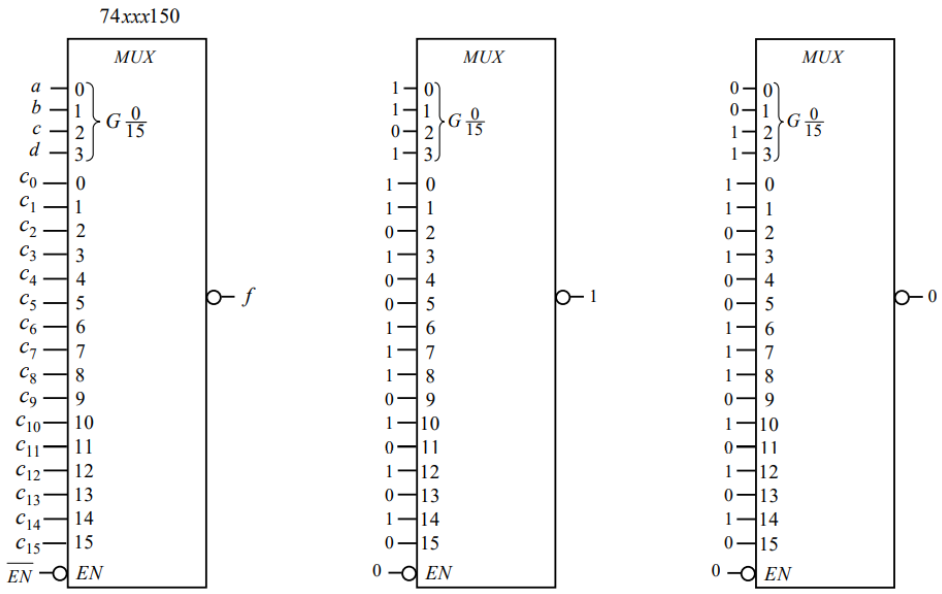
a) Si el estado lógico interno de la entrada EN es igual a 1, entonces el circuito funciona normalmente.
b) Si el estado lógico interno de la entrada EN es igual a 0, entonces las salidas se ponen en el tercer estado, con independencia de los valores que tomen las demás entradas del circuito.

Ejemplos:



Nota: aquí se utiliza la Z para indicar la salida de un tercer estado.

Multiplexores de 16 canales:



Aplicación: Implementación de funciones lógicas con multiplexores:

Cualquier función dependiente de n variables, siempre se puede implementar utilizando:

a) **Opción 1:** Un multiplexor de n entradas de selección (≡ un multiplexor de 2^n canales).
b) **Opción 2:** Un multiplexor de n–1 entradas de selección y 1 puerta NOT.

Algunas funciones dependientes de n variables se pueden implementar utilizando únicamente un multiplexor de n–1 variables de selección (≡ un multiplexor de 2^{n-1} canales).

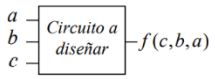
Nota 1: el objetivo es implementar funciones utilizando el menor número posible circuitos integrados (ICs).

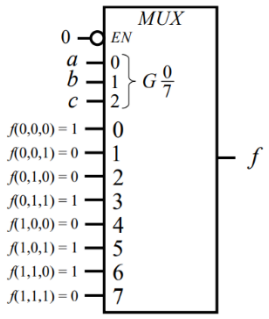
Nota 2: Xilinx utiliza memorias para implementar funciones en sus FPGAs (ver tema 6), mientras que Actel utiliza multiplexores (ver las siguientes diapositivas).

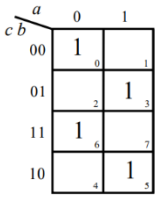
Nota 3: las LUT (look-up table) de las logic cells son las que implementan las funciones lógicas (circuitos combinacionales).

Nota 4: FPGA: field programmable gate array.

Ejemplo: $f(c,b,a)=\sum_3(0,3,5,6)$

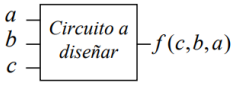


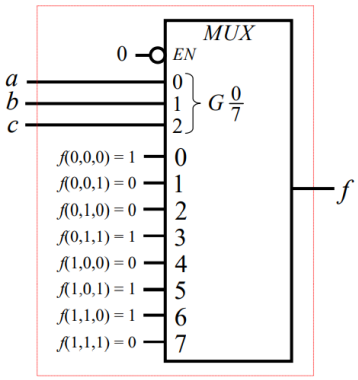




c	b	a	f
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Lo que hay dentro del rectángulo rojo es un circuito combinacional con 3 entradas(c, b y a) y 1 salida (f), que implementa la función : $f(c,b,a)=\sum_3(0,3,5,6)$

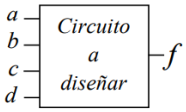


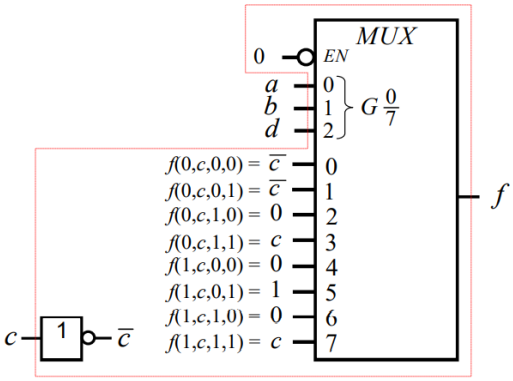


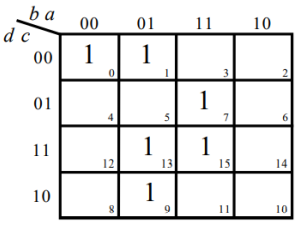
c	b	a	f
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Ejemplo 1: en este caso, los valores que hay que poner en los canales de entrada del multiplexor se determinan utilizando una table de Karnaugh – Veitch.

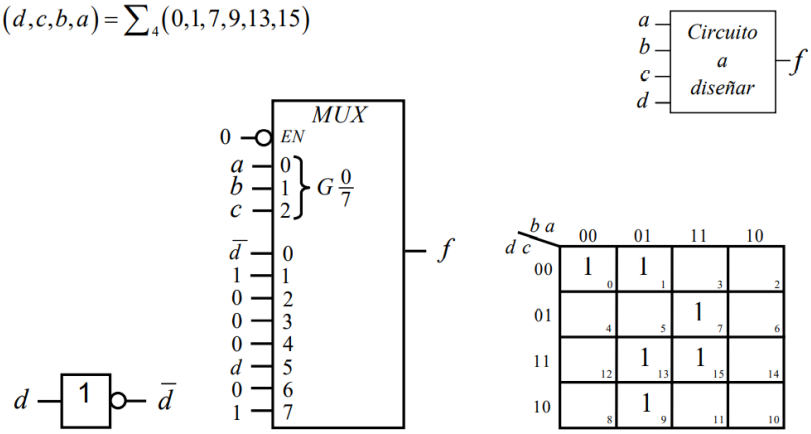
Solución 1: $f(d,c,b,a)=\sum_4(0,1,7,9,13,15)$



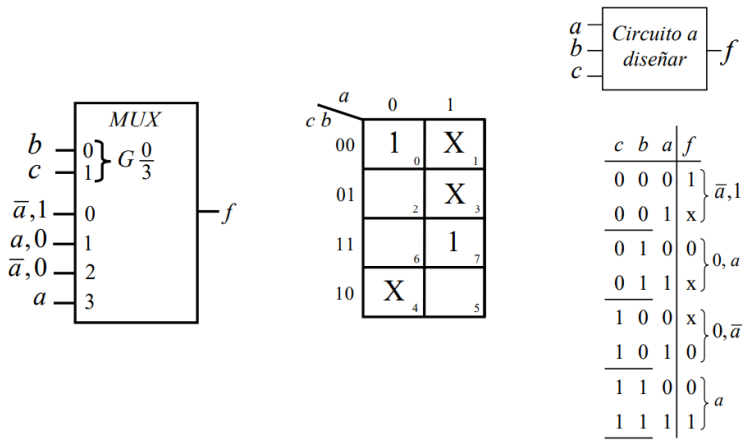




Solución 2: $f(d,c,b,a)=\sum_4(0,1,7,9,13,15)$



Ejemplo 2: la función no esta totalmente definida. $f(c,b,a)=\sum_3(0,7)+\sum_\phi(1,3,4)$



Pregunta: en la práctica, ¿qué valores-señales se ponen en los canales de entrada?

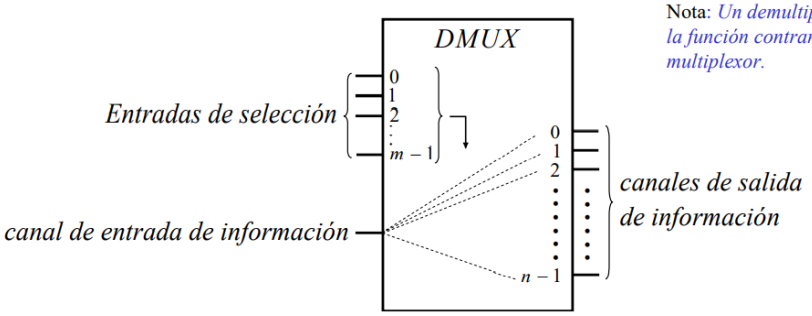
Demultiplexores de n canales (de salida):

Def: Un demultiplexor es un circuito combinacional que tiene:

a) 1 entrada (canal) de información.
b) m entradas de control o de selección.
c) n salidas (canales) de información, cumpliéndose que: $2^m = n$

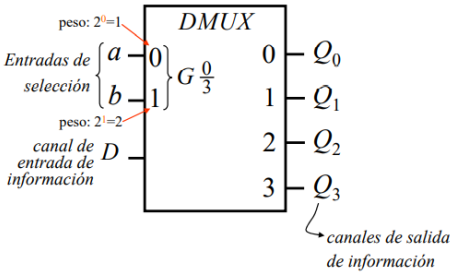
Su funcionamiento se caracteriza por lo siguiente: “La combinación binaria aplicada a las m entradas de control selecciona el canal el canal de salida en el que aparece el valor (1 o 0) presente en el canal de entrada.”

Nota: *Un demultiplexor realiza la función contraria a la de un multiplexor.*



50

Demultiplexor de 4 canales (de salida):



entrada de información — *entradas de selección*

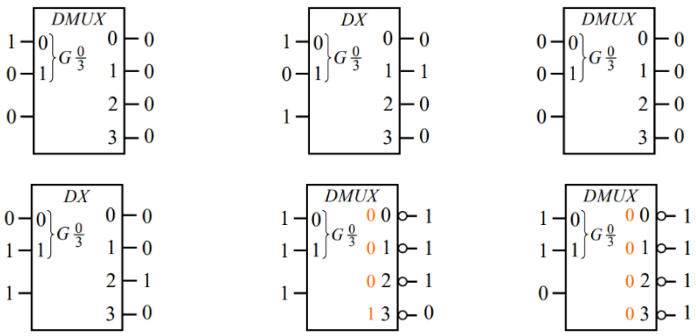
D	b	a	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	0	0
1	0	0	0	0	0	1
0	0	1	0	0	0	0
1	0	1	0	0	1	0
0	1	0	0	0	0	0
1	1	0	0	1	0	0
0	1	1	0	0	0	0
1	1	1	1	0	0	0

|||

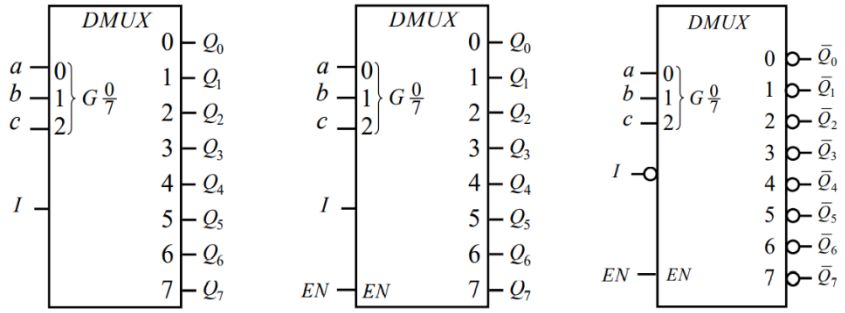
Cuando (b a) = 0 0, en la salida Q_0 hay el valor lógico que haya en la entrada de información D y en las demás salidas hay un 0.

b	a	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	D
0	1	0	0	D	0
1	0	0	D	0	0
1	1	D	0	0	0

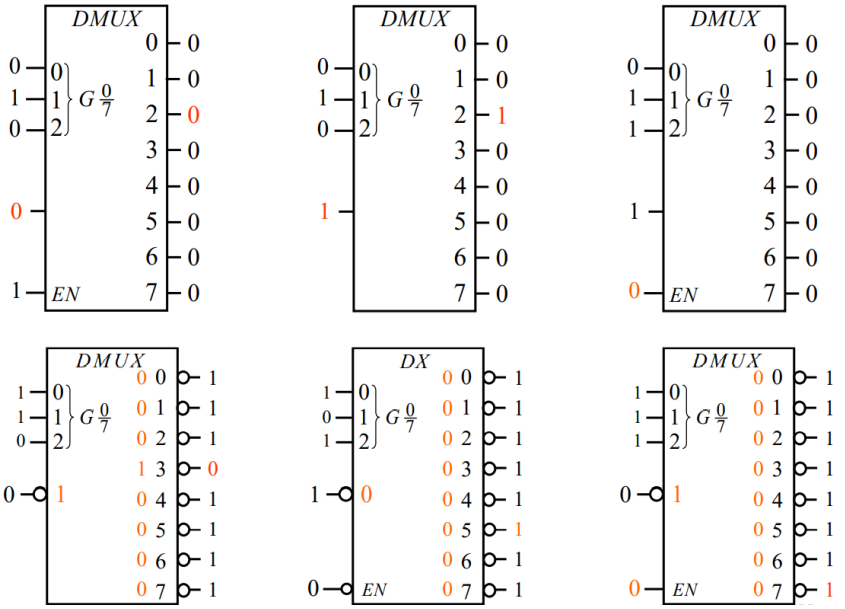
Ejemplos:



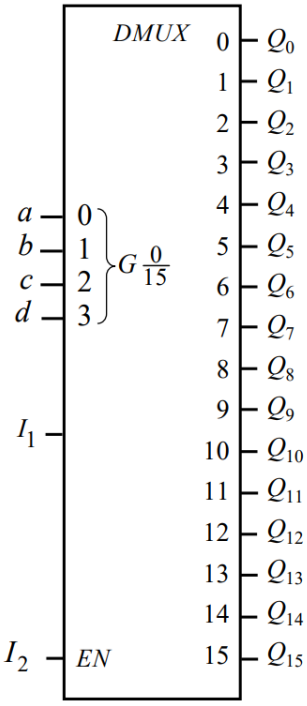
Demultiplexores de 8 canales (de salida):



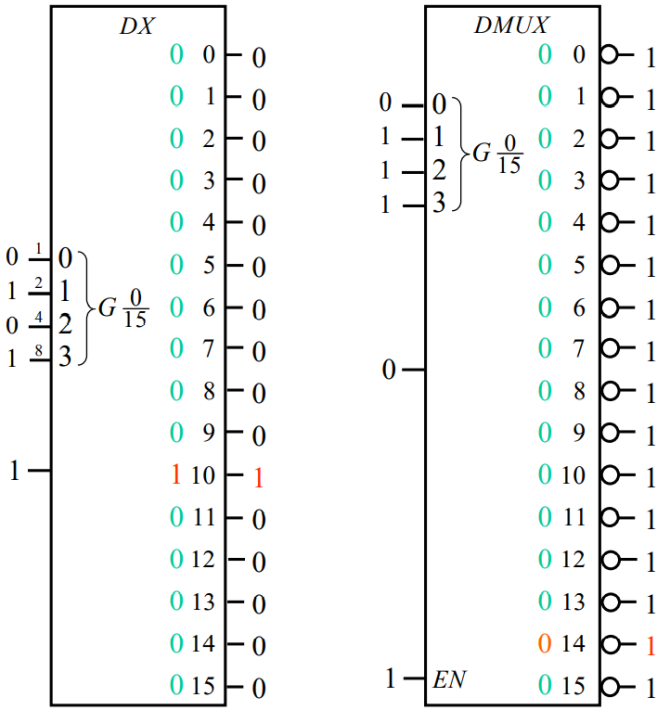
Ejemplos:



Demultiplexores de 16 canales (de salida):

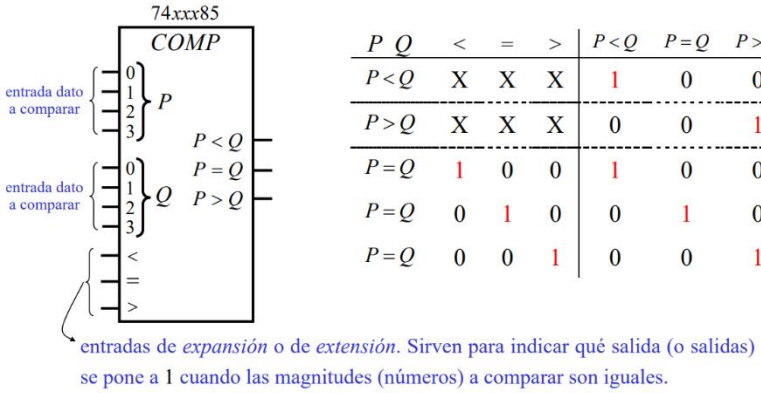


Ejemplos:



Comparadores de magnitud de n bits:

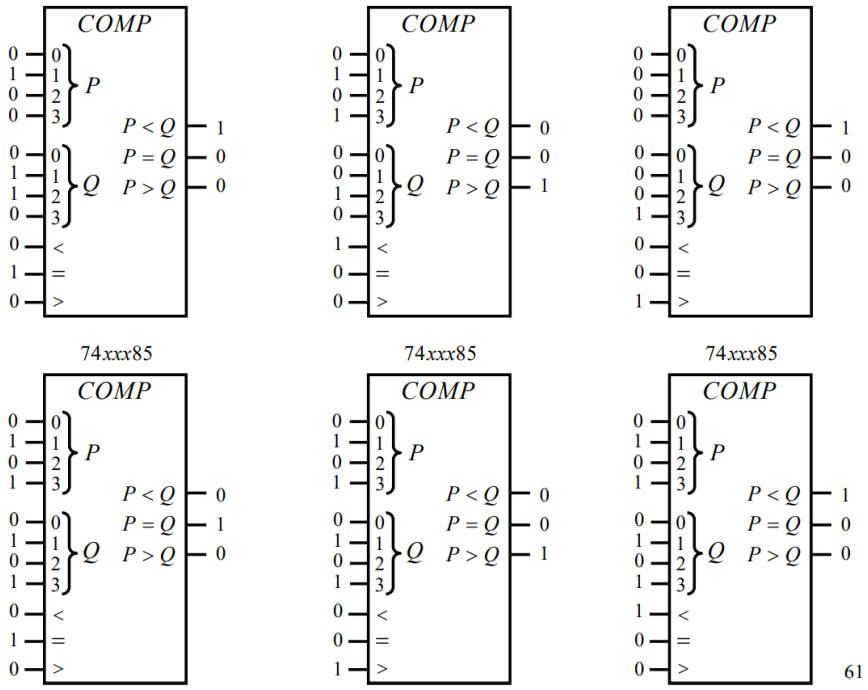
Def: Un comparador de magnitud es un circuito combinacional que indica si dos magnitudes de n bits, representadas en binario natural, son iguales o no, en caso de que no sean iguales, cual de ellas es la mayor.



P Q	<	=	>	P<Q	P=Q	P>Q
P<Q	X	X	X	1	0	0
P>Q	X	X	X	0	0	1
P=Q	1	0	0	1	0	0
P=Q	0	1	0	0	1	0
P=Q	0	0	1	0	0	1

Nota: se comparan magnitudes (números binarios) sin signo.

Ejemplos:



Ejercicio 1: Dados dos números binarios de 4 bits (sin signo), diseñar un circuito que proporcione el mayor de ambos o bien el valor 0 en el caso de que sean iguales. Para resolver este ejercicio no se pueden utilizar puertas lógicas.

Pregunta: ¿Cuantas entradas y cuantas salidas tiene el circuito a diseñar?

Ejercicio 2: Diseñar un circuito al que llegan 3 números binarios A, B y C de 4 bits y una señal de control d. El circuito funciona de la siguiente manera:

a) Si d = 1, entonces el circuito debe indicar la relación entre B y C (B > C, B < C o B = C)
b) Si d = 0, entonces el circuito debe indicar la relación entre B y A (B>A, B<A o B=A) Para resolver este ejercicio no se pueden utilizar puertas lógicas.

Pregunta: ¿ Cuántas entradas y cuántas salidas tiene el circuito a diseñar?

Detectores/generadores de paridad:

Def: Un detector de paridad es un circuito combinacional que detecta/indica si en un grupo de n bits el número de bits que están a 1 es un número par (→ detector de paridad par) o impar (→ detector de paridad impar).

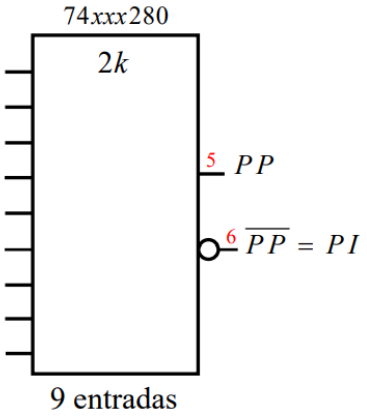
Se cumple que:

$$PP(a_8\,a_7\,a_6\,a_5\,a_4\,a_3\,a_2\,a_1\,a_0) = \overline{a_8 \oplus a_7 \oplus a_6 \oplus a_5 \oplus a_4 \oplus a_3 \oplus a_2 \oplus a_1 \oplus a_0}$$

$$PI(a_8\,a_7\,a_6\,a_5\,a_4\,a_3\,a_2\,a_1\,a_0) = \overline{PP} = a_8 \oplus a_7 \oplus a_6 \oplus a_5 \oplus a_4 \oplus a_3 \oplus a_2 \oplus a_1 \oplus a_0$$

Nota:

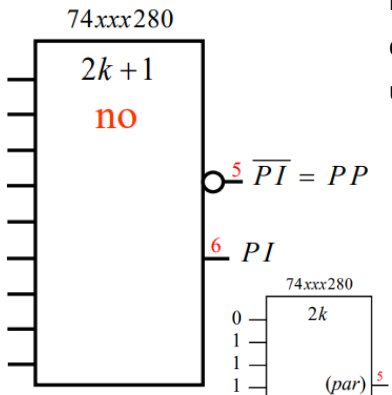
$$x_1 \oplus x_2 \oplus \cdots \oplus x_n = \begin{cases} 1 & \textit{si el número de variables que están a } 1 \textit{ es un número impar.} \\ 0 & \textit{en caso contrario.} \end{cases}$$



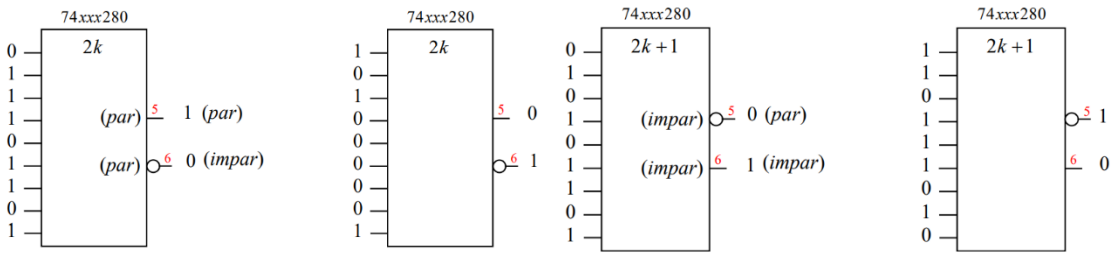
Nota 1: 2k con k∈N siempre es un número par y por lo tanto 2k se utiliza para indicar un detector de paridad par (el 0 se considera un número par).

Nota 2: 2k + 1 con k∈N siempre es un número impar y por lo tanto 2k +1 se utiliza para indicar un detector de paridad impar.

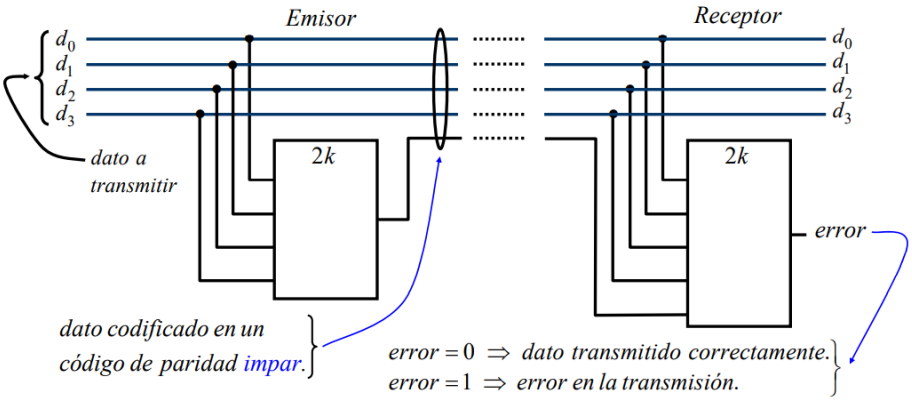
Nota 3: en los circuitos anteriores los estados lógicos internos de las salidas siempre valen lo mismo, sin embargo los estados lógicos externos de las salidas son siempre distintos (uno es el negado del otro). Esto hace que cuando una salida indica paridad par la otra indica paridad impar.



Ejemplos:



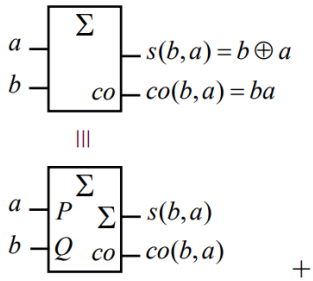
Ejemplo de aplicación:



Curiosidad: los microprocesadores Pentium comprobaban la paridad de la información enviada tanto por el bus de datos como por el bus de direcciones.

Circuitos sumadores: son circuitos combinacionales que realizan sumas aritméticas.

Semisumador de 1 bit: si los valores lógicos (1_s y 0_s) presentes en los terminales de entrada y de salida del circuito indicado a continuación se interpretan como 1_s y 0_s binarios ($1_2 \equiv 1_{lógico}$ y $0_2 \equiv 0_{lógico}$), entonces se puede decir que el circuito indicado a continuación realiza la suma aritmética de dos dígitos binarios (b y a) indicada en la parte derecha.

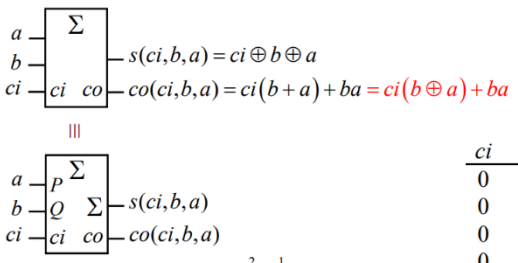


$$\begin{array}{r} b \\ +\,a \\ \hline co\ \ s \end{array} \equiv \begin{array}{r} 0 \\ +\,1 \\ \hline 0\ \ 1 \end{array}$$

b	a	co (2)	s (1)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Nota: se aplica la siguiente analogía: $1_2 = 1_{lógico}$ y $0_2 = 0_{lógico}$.

Sumador total (o completo) de 1 bit (full-adder): el circuito indicado a continuación realizan la suma aritmética de tres dígitos binarios (ci, b, a).



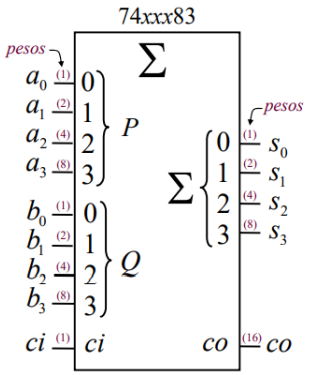
$$\begin{array}{r} ci \\ b \\ +\,a \\ \hline co\ \ s \end{array}$$

$$\begin{array}{r} {}^{2}\ {}^{1} \\ ci \\ b \\ +\,a \\ \hline co\ \ s \end{array} \equiv \begin{array}{r} 1 \\ 0 \\ +\,1 \\ \hline 1\ \ 0 \end{array}$$

ci	b	a	co (2)	s (1)
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Nota: se aplica la siguiente analogía: $1_2 = 1_{lógico}$ y $0_2 = 0_{lógico}$.

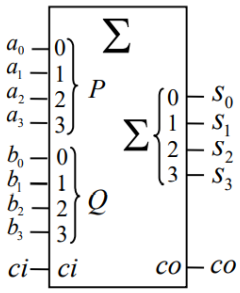
Sumador total de 4 bits: si los valores lógicos (1_s y 0_s) presentes en los terminales de entrada y de salida del circuito indicado a continuación se interpretan como 1_s y 0_s binarios ($1_2 \equiv 1_{lógico}$ y $0_2 \equiv 0_{lógico}$), entonces se puede decir que el circuito indicado a continuación realiza la suma aritmética indicada en la parte derecha.



$$
\begin{array}{rrrrr}
 & & & & ci \\
 & b_3 & b_2 & b_1 & b_0 \\
+ & a_3 & a_2 & a_1 & a_0 \\
\hline
co & s_3 & s_2 & s_1 & s_0
\end{array}
$$

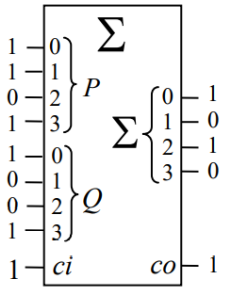
Nota: los nombres asignados a los terminales de entrada y de salida no forman parte del símbolo (sólo se han puesto para poder explicar fácilmente la operación aritmética que realiza el circuito al interpretar los 1s y los 0s lógicos como 1s y 0s binarios respectivamente).

Ejemplo:



$\Longleftrightarrow$

$$
\begin{array}{rrrrr}
 & & & & ci \\
 & b_3 & b_2 & b_1 & b_0 \\
+ & a_3 & a_2 & a_1 & a_0 \\
\hline
co & s_3 & s_2 & s_1 & s_0
\end{array}
$$

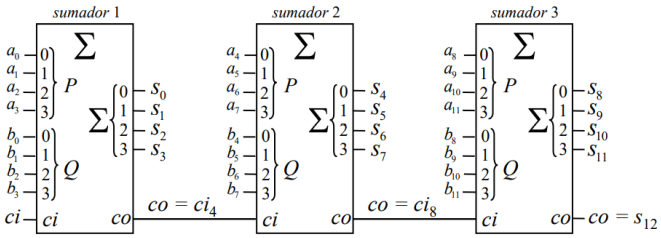


$\Longleftrightarrow$

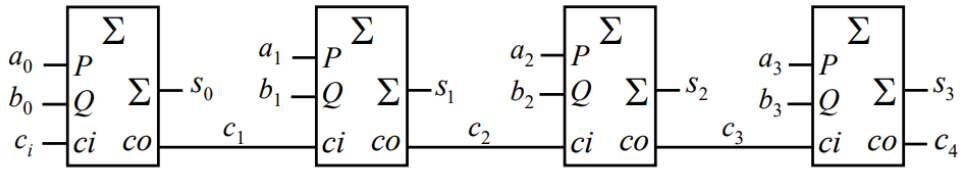
$$
\begin{array}{rrrrr}
 & & & & 1 \\
 & 1 & 0 & 1 & 1 \\
+ & 1 & 0 & 0 & 1 \\
\hline
 & 1 & 0 & 1 & 0 \; 1
\end{array}
$$

Ejemplo de utilidad de la entrada ci (carry input): se efectúa una suma por partes (el co ≡ carry output de una suma pasa a ser el ci ≡ carry input de la siguiente suma).

$$
\begin{array}{rrrrr|rrrr|rrrr}
 & & & & ci_8 & & & & ci_4 & & & & ci \\
 & a_{11} & a_{10} & a_9 & a_8 & a_7 & a_6 & a_5 & a_4 & a_3 & a_2 & a_1 & a_0 \\
+ & b_{11} & b_{10} & b_9 & b_8 & b_7 & b_6 & b_5 & b_4 & b_3 & b_2 & b_1 & b_0 \\
\hline
 & \underbrace{s_{12}\ s_{11}\ s_{10}\ s_9\ s_8}_{sumador\ 3} & & & & \underbrace{s_7\ s_6\ s_5\ s_4}_{sumador\ 2} & & & & \underbrace{s_3\ s_2\ s_1\ s_0}_{sumador\ 1} & & &
\end{array}
$$



Ejemplo de implementación de un sumador total de 4 bits con sumadores totales de 1 bit:



$$s_0 = ci \oplus b_0 \oplus a_0$$

$$c_1 = c_0 b_0 + c_0 a_0 + b_0 a_0$$

$$s_1 = c_1 \oplus b_1 \oplus a_1$$

$$c_2 = c_1 b_1 + c_1 a_1 + b_1 a_1$$

$$s_2 = c_2 \oplus b_2 \oplus a_2$$

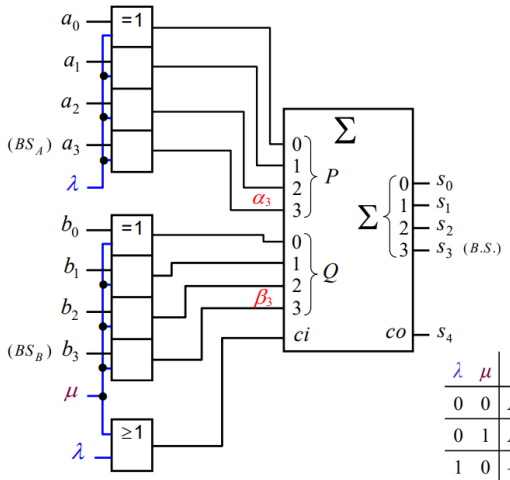
$$c_3 = c_2 \cdot b_2 + c_2 \cdot a_2 + b_2 a_2$$

$$s_3 = c_3 \oplus b_3 \oplus a_3$$

$$c_4 = co = c_3 b_3 + c_3 a_3 + b_3 a_3$$

$$\begin{array}{rrrrr} & & & & ci \\ & b_3 & b_2 & b_1 & b_0 \\ + & a_3 & a_2 & a_1 & a_0 \\ \hline co & s_3 & s_2 & s_1 & s_0 \end{array}$$

Pasatiempos: intenta diseñar un sumador de 1 bit a partir de un semisumador de un bit y del menor número de posible de puertas lógicas básicas.



$$A(\overset{BS}{a_3}\, a_2 a_1 a_0)_{cca2}$$

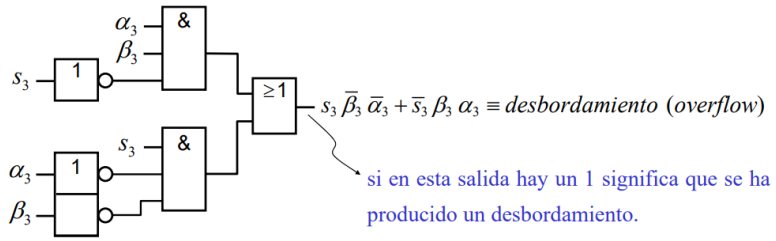
$$B(\overset{BS}{b_3}\, b_2 b_1 b_0)_{cca2}$$

$$S(\overset{BS}{s_3}\, s_2 s_1 s_0)_{cca2}$$

λ	μ	*Operación realizada*
0	0	$A+B$
0	1	$A-B \rightarrow A + ca2(B) = A + ca1(B) + 1$
1	0	$-A+B \rightarrow ca2(A)+B = ca1(A) + B + 1$
1	1	*prohibida*

Nota: μ y λ son entradas auxiliares que permiten especificar la operación a realizar.

Ciruito detector de desbordamientos (overflow).



Recordatorio: ocurre un desbordamiento cuando al sumar dos números positivos se obtiene como resultado un número negativo. También ocurre un

desbordamiento cuando al sumar dos números negativos se obtiene como resultado un número positivo.

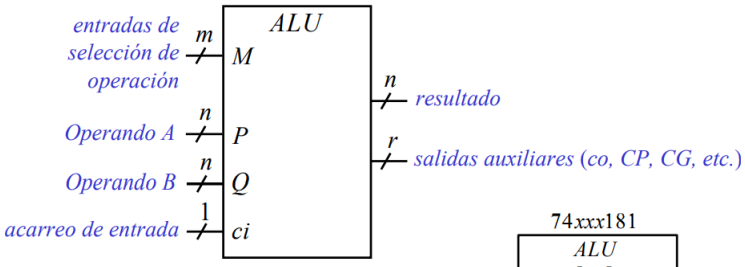
Aplicaciones de los circuitos sumadores:

a) Sumar números binarios.
b) Restar/sumar números en el código cca2.
c) Diseño de convertidores de código.
d) Implementación de comparadores de magnitud.
e) Implementación de detectores/generadores de paridad.
f) Implementación de algunas funciones lógicas.
g) Etc.

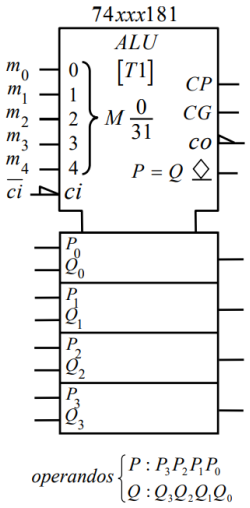
Unidad Aritmético-lógica (ALU: Arithmetic logic unit):

Es un circuito combinacional que puede realizar tanto operaciones aritméticas como lógicas. Las ALUs tienen dos tipos de entradas:

a) Entradas de selección: sirven para indicarle a la ALU la operación (aritmética o lógica) a realizar. El repertorio de instrucciones que puede realizar una ALU se indica en una tabla.
b) Entradas de datos: sirven para proporcionarle a la ALU los datos con los que debe operar (hay operaciones en las que sólo se utiliza un operando).



En las operaciones aritméticas, minus Q representa la la operación ca2.



m_3	m_2	m_1	m_0	Logic $m_4=1$	Arithmetic $m_4=0$
0	0	0	0	$\overline{P}$	P plus ci
0	0	0	1	$\overline{P+Q}$	$P+Q$ plus ci
0	0	1	0	$\overline{P}Q$	$P+\overline{Q}$ plus ci
0	0	1	1	0000	$(-1)_{cca2}$ plus ci
0	1	0	0	$\overline{PQ}$	P plus $(P\cdot\overline{Q})$ plus ci
0	1	0	1	$\overline{Q}$	$P\cdot\overline{Q}$ plus $(P+Q)$ plus ci
0	1	1	0	$P\oplus Q$	$(-1)_{cca2}$ plus P minus Q plus ci
0	1	1	1	$P\cdot\overline{Q}$	$(-1)_{cca2}$ plus $P\cdot\overline{Q}$ plus ci
1	0	0	0	$\overline{P}+Q$	P plus $P\cdot Q$ plus ci
1	0	0	1	$\overline{P\oplus Q}$	P plus Q plus ci
1	0	1	0	Q	$P\cdot Q$ plus $(P+\overline{Q})$ plus ci
1	0	1	1	$P\cdot Q$	$(-1)_{cca2}$ plus $P\cdot Q$ plus ci
1	1	0	0	1111	P plus P plus ci
1	1	0	1	$P+\overline{Q}$	P plus $(P+Q)$ plus ci
1	1	1	0	$P+Q$	P plus $(P+\overline{Q})$ plus ci
1	1	1	1	P	$(-1)_{cca2}$ plus P plus ci

$+ \equiv$ *suma lógica bit a bit* (*bitwise*) *plus / minus* $\equiv$ *suma / resta aritmética*

Ejemplo:

m_4	m_3	m_2	m_1	m_0	*Operación realizada*
0	0	0	0	0	$P\ plus\ ci$
1	0	0	0	0	$\overline{P}$

$P: P_3P_2P_1P_0$

(operación aritmética: $m_4 = 0$ $\left.\begin{matrix} P:1101 \\ ci:1 \end{matrix}\right\} \Rightarrow P\ plus\ ci:\ 1110$ $\equiv 13 + 1 = 14$

(operación aritmética: $m_4 = 1$ $P:1101 \Rightarrow \overline{P}:0010$ $\equiv$ operación bit a bit (*bitwise*)

Notas:

a) + ≡ suma lógica bit a bit (en estas sumas no se producen acarreos y 1+1 = 1).
b) plus/minus ≡ suma/resta aritmética (aquí sí se pueden producir acarreos).
c) las operaciones lógicas se realizan 'bit a bit' (bit-wise).

Ejercicio 3: Diseñar un circuito que calcule la suma de 3 números binarios A = 1011_2, B = 010_2 y C = 100_2. Nota: no se pueden utilizar puertas lógicas.

Ejercicio 4: Diseñar un convertidor de palabras código que en función del valor de una entrada auxiliar α realice las siguientes conversiones:

a) para α = 0 se debe realizar la conversión de $BCD_{natural} \rightarrow BCD_{exceso\ 3}$.
b) para α = 1 se debe realizar la conversión de $BCD_{exceso\ 3} \rightarrow BCD_{natural}$.

Nota: sólo se pueden utilizar bloques funcionales combinacionales y 1 puerta lógica!!!

Ejercicio 5: Diseñar un convertidor de palabras código que en función del valor de una entrada auxiliar α realice las siguientes conversiones:

a) para α = 0 se debe realizar la conversión de $BCD_{exceso\ 3} \rightarrow BCD_{Aiken}$.
b) para α = 1 se debe realizar la conversión de $BCD_{Aiken} \rightarrow BCD_{exceso\ 3}$.

Nota: sólo se pueden utilizar bloques funcionales combinacionales y el menor número posible de puertas XOR.